IconGenAI: Text-to-SVG Icon Generation via Fine-tuning of a Code Language Model

Yauheniya Varabyova
adesso SE
yauheniya.varabyova@adesso.de

April 2026

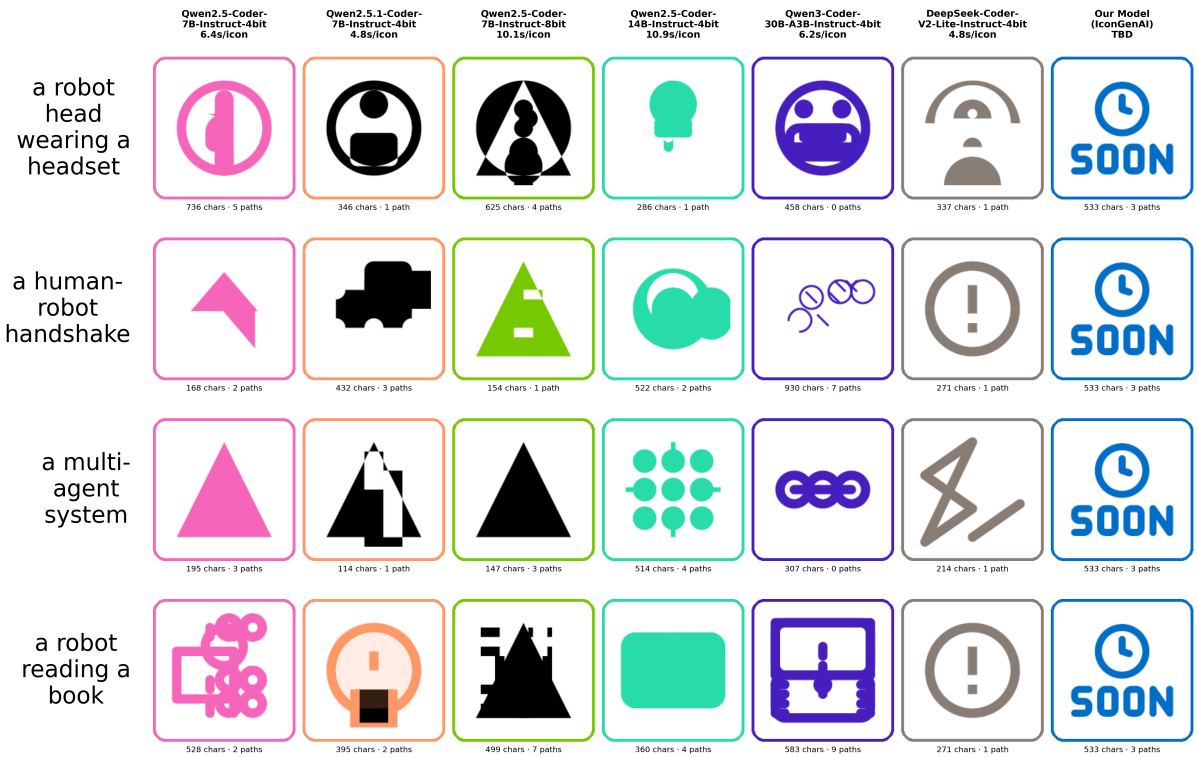

Figure 1: Qualitative comparison of code language models on monochrome SVG icon generation. Rows are prompts; columns are models. Each cell shows the rendered icon with SVG character count and path count below. Our model (IconGenAI) column is a placeholder pending fine-tuning.

Abstract

Scalable Vector Graphics (SVG) is the native format for resolution-independent icons, yet generating high-quality monochrome SVG icons from natural language descriptions remains an unsolved problem. Professional icon design follows a strict convention: compact, single-colour shapes using the `currentColor` CSS property, enabling immediate brand recolouring across any design system without modifying the source file.

Existing text-to-SVG approaches target multi-colour illustrations, logos, and raster-to-vector conversions. None are optimised for the compact, semantically precise, single-colour structure required by production icon workflows — leaving a clear gap for designers and developers who need a lightweight, pixel-perfect icon generator that works across organisations and brands.

We present **IconGenAI**, a text-to-SVG model focused exclusively on monochrome icon generation. Our approach frames icon generation as a structured code generation task and fine-tunes a pre-trained code language model (Qwen3-Coder-30B) using Low-Rank Adaptation (LoRA) on a purpose-built dataset of natural language and SVG icon pairs.

To support this, we introduce the **Iconify-SVG** corpus: 275 912 SVG icons sourced from the Iconify open-source icon ecosystem, filtered by license, normalised to a consistent monochrome format with `currentColor`, and annotated with natural language descriptions generated by a vision language model (Qwen2.5-VL-7B).

[TODO: Fill in: key quantitative results once experiments are complete.]

IconGenAI generates pixel-perfect, immediately renderable, instantly recolourable SVG icons in under one second. Code, model weights, and the Iconify-SVG corpus are released publicly under open-source licenses.

1 Introduction

Icons are the visual vocabulary of modern user interfaces. Every button, menu, notification, and action is accompanied by a small vector graphic that must communicate its meaning instantly and scale perfectly from 16 to 512 pixels. Professional icon libraries — Heroicons, Material Symbols, Tabler, Lucide — contain tens of thousands of carefully crafted SVG files, but they cannot cover the long tail of application-specific concepts: a domain-specific status indicator, a company-branded element, or a novel UI affordance.

Generating icons programmatically from natural language would allow designers and developers to describe what they need — *"a credit card with three sparkles in the top right corner"* — and receive a production-ready SVG file that renders pixel-perfectly at any resolution and remains fully editable in vector design tools.

The gap in existing approaches. Text-to-image diffusion models (Rombach, Blattmann, Lorenz, Esser, and Ommer, 2022; Saharia, Chan, Saxena, et al., 2022) can produce visually plausible icon-like images, but the output is a raster bitmap, not a vector graphic. Converting a raster to SVG via auto-tracing produces imprecise, verbose path data unsuitable for production use. Text-to-SVG methods based on Score Distillation Sampling (Jain, A. Xie, and Abbeel, 2023; Xing et al., 2024) optimize Bézier control points to match a target image, yielding semantically reasonable results for illustrations but generating many redundant paths, lacking the clean single-path-per-shape structure of handcrafted icons, and taking tens of seconds per image.

LLM-based SVG generation (J. A. Rodriguez et al., 2025; Y. Yang et al., 2025; Xing et al., 2025) has recently demonstrated that treating SVG as code leads to more structured, editable outputs. However, these models are trained on general SVG corpora covering logos, diagrams, and illustrations — anime-style artwork, multi-colour paintings, and raster conversions — with no focus on icons.

More critically, *no existing model targets monochrome icon generation specifically.* Professional icon sets — Heroicons, Material Symbols, Tabler, Lucide — use a single semantic colour via the `currentColor` CSS property, allowing icons to be instantly recoloured to match any brand or design system. This monochrome constraint is architecturally distinct from multi-colour SVG generation: the output is structurally simpler (fewer paths, no gradients, no colour layers) but semantically demanding — the silhouette alone must convey the entire meaning of the icon. This focus on *lightweight, pixel-perfect, immediately brandable monochrome icons* is what sets our work apart from all prior text-to-SVG studies.

Our approach. We fine-tune Qwen3-Coder-30B (A. Yang, A. Li, B. Yang, B. Zhang, Hui, B. Zheng, B. Yu, C. Gao, C. Huang, Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, Wei, H. Lin, Jialong Tang, Jian Yang, Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, J. Lin, Dang, Bao, K. Yang, L. Yu, Deng, Mei Li, M. Xue, Mingze Li, P. Zhang, Peng Wang, Q. Zhu, Men, R. Gao, S. Liu, Luo, T. Li, T. Tang, Yin, Xingzhang Ren, X. Wang, Xinyu Zhang, Xuancheng Ren, Fan, Su, Yichang Zhang, Yinger Zhang, Y. Wan, Yuqiong Liu, Z. Wang, Cui, Z. Zhang, Z. Zhou, and Qiu, 2025) — a state-of-the-art mixture-of-experts code language model — on a dataset of 275 912 (SVG icon, natural language description) pairs using LoRA (E. J. Hu, Shen, Wallis, Allen-Zhu, Y. Li, Shean Wang, L. Wang, and W. Chen, 2022). The model is selected from a systematic comparison of Qwen and DeepSeek coder variants (see Figure 1). The natural language descriptions are generated automatically by a vision language model (Qwen2.5-VL-7B (Bai, K. Chen, X. Liu, J. Wang, W. Ge, Song, Dang, Peng Wang, Shijie Wang, Jun Tang, Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, Pengfei Wang, Ding, Fu, Y. Xu, Ye, Xi Zhang, T. Xie, Cheng, Hang Zhang, Z. Yang, Haiyang Xu, and J. Lin, 2025)) applied to rendered PNG images of each icon, following an established automated annotation pipeline (J. A. Rodriguez et al., 2025; Xing et al., 2025).

The resulting model, **IconGenAI**, generates valid, renderable, monochrome SVG icons in under one second. Unlike diffusion-based approaches, the output is natively editable, consistently structured, uses

`currentColor` for immediate brand recolouring, and follows the conventions of professional icon design and development workflows.

Contributions.

1. **The Iconify-SVG Corpus.** A curated dataset of 275 912 SVG icons from the Iconify open-source ecosystem, filtered by license (excluding ShareAlike and GPL variants), normalized to a consistent SVG format, and annotated with VLM-generated natural language descriptions. To our knowledge, this is the first publicly documented use of Iconify as a machine learning training corpus.

2. **An automated icon annotation pipeline.** A reproducible pipeline for rendering SVG icons and generating structured short and long natural language descriptions using a vision language model, with documented prompt engineering and quality metrics.

3. **IconGenAI.** A monochrome text-to-SVG model fine-tuned with LoRA on Qwen3-Coder-30B, trained specifically for single-colour icon generation with curriculum ordering (simple to complex) and multi-variant instruction augmentation. The monochrome `currentColor` constraint makes outputs immediately usable across any brand or design system.

4. **An icon-specific evaluation framework.** Metrics tailored to icon generation quality: SVG validity rate, rendering fidelity, semantic alignment (CLIP score), and structural economy (path count), together with a held-out test set of 13 795 icons for reproducible benchmarking.

Code, model weights, and the Iconify-SVG corpus are released at: `https://github.com/TODO/icon-genai` under open-source licenses.

2 Related Work

2.1 Foundational Vector Graphics Generation

Early work on learned vector graphics generation treated SVG as a sequence of drawing commands. Gontijo Lopes, Ha, Eck, and Shlens (2019) introduced SVG-VAE, a class-conditioned variational autoencoder operating on sequential LSTM representations of Bézier curve commands, trained on 14 million font characters. While limited to simple, single-glyph shapes, SVG-VAE established the command-sequence paradigm that most subsequent work adopts.

Carlier, Danelljan, Alahi, and Timofte (2020) extended this with DeepSVG, a hierarchical Transformer that disentangles high-level shape identity from low-level path commands. DeepSVG introduced a large-scale SVG icon dataset and demonstrated smooth interpolation and animation of icon sequences, establishing the icon domain as a productive testbed for vector generation research.

2.2 Diffusion-Based Text-to-SVG

The dominant paradigm from 2023–2024 applies Score Distillation Sampling (SDS) to optimize Bézier path parameters guided by a pretrained text-to-image diffusion model. VectorFusion (Jain, A. Xie, and Abbeel, 2023) demonstrated that a differentiable rasterizer (DiffVG) can bridge the pixel and vector domains, allowing gradients from Stable Diffusion to directly update SVG control points. SVGDreamer (Xing et al., 2024) improved upon this with Variational Score Distillation and semantic-driven path initialization, addressing the over-smoothing and color saturation issues endemic to SDS-based approaches.

These methods produce visually plausible results for illustrations and logos but are poorly suited to icon generation for three reasons: (1) generation takes 30–120 seconds per image; (2) the optimized paths do not respect icon design conventions; and (3) the resulting SVG is typically an over-parameterized approximation rather than a compact, editable structure.

2.3 LLM-Native SVG Code Generation

A parallel line of work treats SVG generation as code generation and leverages the sequence modeling capabilities of large language models. This is the paradigm our work follows.

IconShop (R. Wu et al., 2023) tokenizes SVG path data into a discrete vocabulary and trains an autoregressive Transformer on a curated icon corpus. It supports text-guided generation, interpolation

between icons, and semantic combination. While closely related to our approach, IconShop uses a purpose-built tokenization scheme and is not built on a pretrained code language model.

StarVector (J. A. Rodriguez et al., 2025) treats image-to-SVG vectorization as code generation using a Vision-Language Model backbone. Trained on the 2M-sample SVG-Stack corpus, it handles diverse SVG types including icons, logos, and technical diagrams. LLM4SVG (Xing et al., 2025) introduces 55 domain-specific SVG semantic tokens and trains on 250K SVGs with 580K instruction pairs, demonstrating that structured tokenization improves generation quality.

OmniSVG (Y. Yang et al., 2025) unifies image-conditioned and text-conditioned generation via a Qwen2.5-VL backbone and the 2M-sample MMSVG-2M dataset. SVGen (F. Wang, Zhao, Yuandong Liu, D. Zhang, J. Gao, Sun, and X. Li, 2025) applies Chain-of-Thought reasoning and Group Relative Policy Optimization (GRPO) reinforcement learning, showing that explicit reasoning steps significantly improve structural validity.

Distinction from our work. All the above LLM-based methods are trained on general SVG corpora and evaluated on diverse SVG types. We focus exclusively on the icon domain, which admits a narrower distribution (single-color fills, 1–20 paths, 16–512px viewBox) and therefore benefits from a specialized training set. We build the first ML training corpus from the Iconify ecosystem and perform domain-specific fine-tuning on Qwen3.5-9B, targeting the quality and structural conventions expected of production icons.

2.4 Automated SVG Captioning and Datasets

Several recent works construct large SVG training datasets using automated annotation pipelines. StarVector (J. A. Rodriguez et al., 2025) renders SVG files to images and uses a VLM to generate text descriptions, assembling the SVG-Stack corpus. LLM4SVG (Xing et al., 2025) similarly uses automated pipelines to produce 580K SVG-text instruction pairs from 250K SVGs. SVGen (F. Wang, Zhao, Yuandong Liu, D. Zhang, J. Gao, Sun, and X. Li, 2025) applied this to approximately 500K icons from the Iconfont repository to produce the SVG-1M dataset.

Our Iconify-SVG corpus applies the same automated annotation strategy to the Iconify ecosystem. The key differences are: (1) a rigorous per-set license filter preserving 275 912 icons under permissive open-source terms; (2) use of structured short-and-long caption pairs tuned for instruction following rather than single descriptions; (3) full documentation of the normalization pipeline to ensure reproducibility.

2.5 Evaluation of SVG Generation

SVGenius (S. Chen, Dong, Haolei Xu, X. Wu, F. Tang, Hang Zhang, Yan, L. Wu, W. Zhang, Hou, et al., 2025) benchmarks 22 LLMs on SVG understanding, editing, and generation tasks, finding that reasoning-enhanced models outperform pure scaling and that quality degrades sharply with rising SVG complexity. VectorGym (J. Rodriguez, Haotian Zhang, Puri, T. Zhang, Pramanik, M. Lin, X. Xie, Terral, Kaushik, Shariff, Taslakian, Gella, Rajeswar, Vazquez, Pal, and Pedersoli, 2026) provides human-annotated benchmarks across four SVG tasks. Both works confirm the need for domain-specific evaluation metrics beyond standard image quality scores. Our evaluation framework adapts these insights to the icon domain.

3 The Iconify-SVG Corpus

3.1 Source: The Iconify Ecosystem

Iconify[1] is an open-source meta-library aggregating over 300,000 icons from 225 icon sets. Unlike individual icon libraries, Iconify provides a unified JSON format and a public GitHub repository (`iconify/icon-sets`) containing all icons as structured data without requiring web scraping. Each set carries explicit license metadata, making per-set license filtering straightforward.

3.2 License Filtering

Machine learning training use is governed by the license terms of the training data. We apply a conservative license filter that retains only sets under permissive open-source licenses and excludes all ShareAlike and copyleft variants, as detailed in Table 1.

[1] `https://iconify.design`

Table 1: License filter applied to Iconify collections. ShareAlike and GPL licenses are excluded due to their potential implications for model weight distribution.

License (SPDX)	Sets	Icons	Decision
MIT	99	114,830	Include
Apache-2.0	30	82,644	Include
CC-BY-4.0	51	55,648	Include
CC0-1.0	8	8,051	Include
OFL-1.1	13	6,211	Include
CC-BY-3.0	1	4,123	Include
ISC	3	2,998	Include
CC-BY-NC-4.0	1	479	Include (non-commercial)
Unlicense	1	430	Include
MPL-2.0	1	288	Include
BSD-3-Clause	1	210	Include
CC-BY-SA-4.0	7	21,828	Exclude (ShareAlike)
CC-BY-NC-SA-4.0	1	1,577	Exclude (ShareAlike)
CC-BY-SA-3.0	2	79	Exclude (ShareAlike)
GPL-*	6	1,522	Exclude (Copyleft)
Total included	**209**	**275,912**	

Additionally, 9,045 icons marked as `hidden` in the Iconify metadata (deprecated or internal entries) are excluded, and alias entries (icons defined as references to parent icons) are skipped to avoid duplicate training samples.

This study is conducted for non-commercial research purposes. Beyond license terms, we note that (1) the EU DSM Directive (Article 4) explicitly permits text and data mining of lawfully accessed content for research use; (2) the trained model weights are a derivative work that does not reproduce training icons verbatim. Each record in the released dataset retains the original Iconify metadata fields `collection_name`, `license_spdx`, `author`, and `license_url`, enabling downstream users to comply with the attribution requirements of CC-BY and similar licenses.

3.3 SVG Normalization

Raw Iconify data consists of SVG `body` fragments — the inner content of the `<svg>` element without the root tag or namespace declaration. We assemble each icon into a complete, valid SVG document:

Listing 1: Assembled SVG document format

```
<svg xmlns="http://www.w3.org/2000/svg"
     viewBox="0 0 {W} {H}" width="{W}" height="{H}">
  {body}
</svg>
```

Two color conventions appear in the corpus. *Monochrome* icons (the majority) use `currentColor` as the fill and stroke value — the standard convention in icon design that allows the icon to inherit its context's text color. *Multicolor* icons contain explicit color values (hex codes, named colors, or `rgb()` expressions) hardcoded in the path elements, e.g. `fill="#4285F4"`. Each record carries an `is_multicolor` boolean flag computed by scanning the SVG body for explicit color values (hex codes, `rgb()`, or the named colors `white`/`black`) outside any `currentColor` reference. This flag is used during curriculum ordering (Section 4.5) to expose the model to simple monochrome icons before the more complex multicolor ones.

3.4 Automated Caption Generation

Natural language descriptions are generated using Qwen2.5-VL-7B-Instruct (Bai, K. Chen, X. Liu, J. Wang, W. Ge, Song, Dang, Peng Wang, Shijie Wang, Jun Tang, Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, Pengfei Wang, Ding, Fu, Y. Xu, Ye, Xi Zhang, T. Xie, Cheng, Hang Zhang, Z. Yang, Haiyang Xu, and J. Lin, 2025), a vision language model running on Apple Silicon via the MLX framework (Hannun, Digani,

Katharopoulos, and Collobert, 2023). Each icon is rendered to a 256×256 PNG using CairoSVG and passed to the VLM with a structured prompting template.

We generate two caption variants per icon:

- **Short caption (5–12 words):** a natural search query expressing the concept and visual style, e.g., *"outlined credit card with sparkles"* or *"filled shopping cart with checkmark badge"*. This serves as the primary training prompt.
- **Long caption (20–35 words):** a detailed description including visual style (outline/filled/flat/solid/line), all visible elements, any decorations or modifiers, and likely UI context.

The icon name and collection name from Iconify metadata are provided as context in the prompt, helping the model identify the semantic intent of ambiguous icons. Caption generation runs checkpointed at the icon level, enabling resumable processing across multiple sessions.

[TODO: Report: total captioning time, model throughput, fraction of icons requiring fallback to icon name (parse failure rate).]

3.5 Dataset Statistics

[TODO: Add figures once captioning is complete: (1) distribution of path_count (histogram); (2) proportion monochrome vs. multicolor by collection; (3) caption length distribution (short and long); (4) most frequent concepts in short captions (word cloud or bar chart); (5) sample grid of icons across collections.]

Table 2 summarises the key characteristics of the Iconify-SVG corpus.

Table 2: Iconify-SVG corpus statistics.

Attribute	Value
Total icons	275,912
Icon sets	209
Monochrome icons	**[TODO: X%]**
Multicolor icons	**[TODO: X%]**
Median path count	**[TODO: X]**
Mean path count	**[TODO: X]**
Mean short caption length (words)	**[TODO: X]**
Mean long caption length (words)	**[TODO: X]**
Training split	248,120 (90%)
Validation split	13,896 (5%)
Test split	13,896 (5%)

4 Method

4.1 SVG as Code

SVG is an XML-based language for describing two-dimensional vector graphics. An icon SVG typically consists of a root `<svg>` element with a `viewBox` attribute defining the coordinate space, followed by one or more geometric primitives: `<path>` elements with `d` attributes encoding sequences of drawing commands (MoveTo, LineTo, CubicBézier, EllipticalArc, ClosePath), and optionally `<circle>`, `<rect>`, `<polygon>`, or `<g>` grouping elements.

This structure is naturally representable as a text sequence, and modern code language models pre-trained on source code corpora have already seen XML and SVG-like content in their training data. We leverage this prior knowledge by treating icon generation as instruction-following code generation: given a natural language description as the user instruction, produce a complete, valid SVG document as the assistant response.

4.2 Base Model

We use Qwen3.5-9B (A. Yang, A. Li, B. Yang, B. Zhang, Hui, B. Zheng, B. Yu, C. Gao, C. Huang, Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, Wei, H. Lin, Jialong Tang, Jian Yang, Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, J. Lin, Dang, Bao, K. Yang, L. Yu, Deng, Mei Li, M. Xue, Mingze Li, P. Zhang, Peng Wang, Q. Zhu, Men, R. Gao, S. Liu, Luo, T. Li, T. Tang, Yin, Xingzhang Ren, X. Wang, Xinyu Zhang, Xuancheng Ren, Fan, Su, Yichang Zhang, Yinger Zhang, Y. Wan, Yuqiong Liu, Z. Wang, Cui, Z. Zhang, Z. Zhou, and Qiu, 2025) as the base model. Qwen3.5 represents the current generation of Qwen language models (A. Yang, B. Yang, B. Zhang, Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, Wei, H. Lin, Jian Yang, Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, J. Lin, Dang, Lu, Bao, K. Yang, L. Yu, Mei Li, M. Xue, P. Zhang, Q. Zhu, Men, R. Lin, T. Li, T. Tang, Xia, Xingzhang Ren, Xuancheng Ren, Fan, Su, Yichang Zhang, Y. Wan, Yuqiong Liu, Cui, Z. Zhang, and Qiu, 2025), featuring a 128K token context window (important for complex SVGs), strong performance on code generation benchmarks, and native instruction-following capabilities through the ChatML prompt format. The 9B parameter scale balances generation quality with the memory and compute constraints of local Apple Silicon hardware.

4.3 Instruction-Tuning Formulation

Each training example is formatted as a three-turn conversation in the ChatML format used by Qwen3.5:

Listing 2: Training record format (ChatML)

```
{"messages": [
  {"role": "system",
   "content": "You␣are␣an␣SVG␣icon␣generator.␣Given␣a␣description,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣output␣clean,␣valid␣SVG␣code␣for␣a␣single␣icon.␣Use
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣currentColor␣for␣fill␣and␣stroke.␣Output␣only␣the
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣SVG␣element,␣no␣explanation."},
  {"role": "user",
   "content": "Generate␣an␣SVG␣icon:␣outlined␣credit␣card␣with␣sparkles"},
  {"role": "assistant",
   "content": "<svg␣xmlns=...␣viewBox=...><path␣.../></svg>"}
]}
```

4.4 Data Augmentation

For each icon we generate up to three training variants with different user-side prompts pointing to the same SVG target: (1) the short VLM caption (*"outlined credit card with sparkles"*), (2) the long VLM caption (full description with style and context), and (3) the raw icon name from the Iconify metadata (*"credit-card"*). This teaches the model to handle prompts of varying specificity and phrasing, reflecting the diversity of real-world user queries.

4.5 Curriculum Learning

Following F. Wang, Zhao, Yuandong Liu, D. Zhang, J. Gao, Sun, and X. Li (2025), training examples are ordered from simple to complex using a two-level sort key: (1) monochrome icons before multicolor, (2) ascending path count within each group. This curriculum strategy exposes the model to the dominant icon patterns first, grounding its SVG priors before introducing the long-tail of complex multicolor icons.

4.6 LoRA Fine-tuning

We apply Low-Rank Adaptation (E. J. Hu, Shen, Wallis, Allen-Zhu, Y. Li, Shean Wang, L. Wang, and W. Chen, 2022) to fine-tune Qwen3.5-9B, keeping the pre-trained weights frozen and adding trainable low-rank matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ at each attention and MLP projection layer. The effective weight update is:

$$W' = W + \alpha \cdot AB$$

where r is the rank (we use $r = 64$) and α is a scaling factor. With 4-bit quantization of the frozen base weights (QLoRA), total GPU memory is under 16 GB, enabling training on Apple M4 hardware via the MLX framework (Hannun, Digani, Katharopoulos, and Collobert, 2023).

Training configuration. **[TODO: Fill in after training: batch size, learning rate, number of iterations, hardware, training time per epoch.]** Table 3 summarises the training hyperparameters.

Table 3: Training configuration for IconGenAI.

Hyperparameter	**Value**
Base model	Qwen3.5-9B-Instruct
Quantization	4-bit (QLoRA)
LoRA rank r	64
LoRA α	128
LoRA target layers	attention + MLP projections
Training epochs	2–3
Batch size	**[TODO: X]**
Learning rate	**[TODO: X]**
Training records	**[TODO: X]** (with augmentation)
Hardware	Apple M4 / **[TODO: cloud GPU]**
Training time	**[TODO: X hours]**

5 Experiments

[TODO: This section is a template. Fill in all tables and results after training and evaluation are complete.]

5.1 Evaluation Metrics

We evaluate on the held-out test split of 13,896 icons. For each test icon, the model receives the short VLM caption as the prompt and generates SVG code. We measure the following:

- **SVG Validity Rate (VR):** fraction of generated outputs that parse as well-formed XML with a valid root `<svg>` element. A model that generates syntactically broken SVG is unusable.
- **Rendering Success Rate (RSR):** fraction of valid SVGs that render without errors using CairoSVG. Catches semantic issues such as malformed `d` attributes that parse as XML but fail to render.
- **CLIP Score:** cosine similarity between the CLIP (Radford, Kim, Hallacy, et al., 2021) embedding of the rendered icon PNG and the CLIP embedding of the text prompt, measuring semantic alignment. We use ViT-B/32 (Radford, Kim, Hallacy, et al., 2021) as the backbone.
- **Mean Path Count (MPC):** average number of `<path>` elements in generated icons. Lower is preferable; professional icons typically use 1–5 paths. This measures structural economy.
- **FID (Fréchet Inception Distance):** distributional similarity between rendered generated icons and rendered ground-truth icons, computed over the test split. Lower is better.

5.2 Baselines

We compare against two categories of baseline. *Zero-shot LLM baselines* are general-purpose language models prompted with the same system and user messages as IconGenAI, but without any SVG-specific fine-tuning; the "(zero-shot)" label distinguishes these from models that were explicitly trained for SVG generation. *Supervised SVG baselines* are models trained end-to-end on SVG corpora and represent the current open-weight state of the art.

- **Qwen3.5-9B (zero-shot):** the base model without fine-tuning, prompted identically to IconGenAI. Isolates the contribution of domain-specific fine-tuning.
- **GPT-5.4 (zero-shot):** OpenAI's GPT-5.4 prompted via the API with the same prompts. Serves as a strong commercial LLM reference point. Parameter count is not publicly disclosed.

- **StarVector-8B** (J. A. Rodriguez et al., 2025): an 8B-parameter open-weight model that frames SVG generation as code generation from a vision-language backbone; state of the art on general SVG vectorization at the time of writing.
- **OmniSVG-8B** (Y. Yang et al., 2025): 8B-parameter model trained on MMSVG-2M including a dedicated icon subset, making it the most directly comparable supervised baseline.

5.3 Quantitative Results

[TODO: Fill in Table 4 once evaluation is complete.]

Table 4: Main results on the Iconify-SVG test split. Best results in **bold**.

Model	Params	VR (%)	RSR (%)	CLIP ↑	MPC ↓
Commercial zero-shot					
GPT-5.4	n/a	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
Open-weight zero-shot					
Qwen3.5-**[TODO: 1.7B or 9B]**	**[TODO: 1.7B or 9B]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
SVG-specific (supervised)					
StarVector-8B	8B	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
OmniSVG-8B	8B	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
IconGenAI (ours)	**[TODO: 1.7B or 9B]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

5.4 Ablation Studies

[TODO: Run and fill in Table 5 once training is complete.]

Table 5: Ablation study on key design decisions.

Variant	VR (%)	RSR (%)	CLIP ↑	FID ↓
IconGenAI (full)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
w/o curriculum	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
w/o augmentation	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
short captions only	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
1 epoch (vs. 3)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

6 Analysis

6.1 Qualitative Examples

[TODO: Insert a figure grid: 3–4 rows of (prompt, generated SVG, ground-truth icon) showing representative successes across icon styles. Row 1: simple monochrome outline icons (e.g., arrows, UI actions) Row 2: multicolor icons or icons with badges/decorators Row 3: abstract/metaphorical icons (e.g., "speed", "security") Row 4: multi-element compositions]

[TODO: Insert figure: qualitative_examples.pdf]

Figure 2: Representative IconGenAI outputs alongside their prompts and the closest ground-truth icons from the test split.

6.2 Failure Modes

[TODO: Analyse and describe after evaluation. Expected failure cases based on prior work and dataset properties:]

- **Rare or abstract concepts** — icons for uncommon concepts (e.g., domain-specific technical symbols) that appear infrequently in training may produce generic or semantically misaligned outputs.
- **Complex multi-element compositions** — icons described as having more than 3–4 distinct visual elements tend to have lower rendering fidelity, consistent with the SVGenius finding that quality degrades with rising SVG complexity.
- **Invalid path data** — occasionally the model generates syntactically plausible but geometrically malformed `d` attributes (e.g., missing coordinate pairs after a command letter). This manifests as rendering errors. **[TODO: Quantify rate.]**
- **ViewBox mismatch** — some generated icons place content outside the declared viewBox, resulting in clipped rendering.

6.3 Caption Quality Ablation

[TODO: Compare: (1) VLM-generated captions vs. (2) raw icon names as prompts for fine-tuning. This isolates the contribution of the captioning pipeline.]

6.4 Comparison with General SVG Models

[TODO: Discuss qualitative differences between IconGenAI and StarVector/OmniSVG on icon-specific prompts. Hypothesis: general models produce more complex, less icon-like SVG structure; IconGenAI produces compact, `currentColor` convention-following icons consistent with professional icon libraries.]

7 Conclusion

[TODO: Write after experiments are complete. Draft outline:]

We presented IconGenAI, a domain-specific language model for text-to-SVG icon generation. We introduced the Iconify-SVG corpus — 275 912 license-filtered, normalized SVG icons from the Iconify ecosystem, annotated with VLM-generated natural language descriptions — and demonstrated that LoRA fine-tuning of Qwen3.5-9B on this corpus yields a model that generates valid, renderable, semantically aligned SVG icons from natural language prompts.

[TODO: Insert key numerical finding.]

Limitations. The Iconify-SVG corpus reflects the distribution of the Iconify ecosystem: predominantly single-color, geometric icons in flat or outline styles. The model may generalize poorly to photorealistic SVGs, complex illustrations, or icon styles not represented in Iconify. Caption quality depends on the vision model and prompt template; a small fraction of icons receive semantically inaccurate descriptions that may reduce training signal quality.

Future work. We identify three promising directions. First, incorporating reasoning-enhanced training (F. Wang, Zhao, Yuandong Liu, D. Zhang, J. Gao, Sun, and X. Li, 2025; Xing, Z. Xue, Guan, Jing Zhang, D. Xu, and Q. Yu, 2026) via GRPO reinforcement learning may further improve structural validity without increasing dataset size. Second, interactive editing — given an existing icon and a textual modification instruction, produce an updated SVG — is a natural extension enabled by the instruction-following formulation. Third, evaluating the model in a user study with designers and front-end developers would provide ecological validity beyond the automated metrics used here.

References

Bai, Shuai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin (2025). *Qwen2.5-VL Technical Report*. arXiv: `2502.13923 [cs.CV]`. URL: `https://arxiv.org/abs/2502.13923`.

Carlier, Alexandre, Martin Danelljan, Alexandre Alahi, and Radu Timofte (2020). "DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2007.11301.

Chen, Siqi, Xinyu Dong, Haolei Xu, Xingyu Wu, Fei Tang, Hang Zhang, Yuchen Yan, Linjuan Wu, Wenqi Zhang, Guiyang Hou, et al. (Oct. 2025). "SVGenius: Benchmarking LLMs in SVG Understanding, Editing and Generation". In: *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*. ACM, pp. 13289–13296. DOI: 10.1145/3746027.3758287. URL: https://arxiv.org/abs/2506.03139.

Gontijo Lopes, Raphael, David Ha, Douglas Eck, and Jonathon Shlens (2019). "A Learned Representation for Scalable Vector Graphics". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. URL: https://arxiv.org/abs/1904.02632.

Hannun, Awni, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert (2023). *MLX: An Array Framework for Apple Silicon*. URL: https://github.com/ml-explore/mlx.

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2106.09685.

Jain, Ajay, Amber Xie, and Pieter Abbeel (2023). "VectorFusion: Text-to-SVG by Abstracting Pixel-Based Diffusion Models". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2211.11319.

Radford, Alec, Jong Wook Kim, Chris Hallacy, et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision". In: *Proceedings of the International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2103.00020.

Rodriguez, Juan, Haotian Zhang, Abhay Puri, Tianyang Zhang, Rishav Pramanik, Meng Lin, Xiaoqing Xie, Marco Terral, Darsh Kaushik, Aly Shariff, Perouz Taslakian, Spandana Gella, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli (2026). *VectorGym: A Multitask Benchmark for SVG Code Generation, Sketching, and Editing*. arXiv: 2603.29852 [cs.GR]. URL: https://arxiv.org/abs/2603.29852.

Rodriguez, Juan A. et al. (2025). "StarVector: Generating Scalable Vector Graphics Code from Images and Text". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2312.11556.

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer (2022). "High-Resolution Image Synthesis with Latent Diffusion Models". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2112.10752.

Saharia, Chitwan, William Chan, Saurabh Saxena, et al. (2022). "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2205.11487.

Wang, Feiyu, Zhiyuan Zhao, Yuandong Liu, Da Zhang, Junyu Gao, Hao Sun, and Xuelong Li (Oct. 2025). "SVGen: Interpretable Vector Graphics Generation with Large Language Models". In: *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*. ACM, pp. 9608–9617. DOI: 10.1145/3746027.3755011. URL: http://dx.doi.org/10.1145/3746027.3755011.

Wu, Ronghuan et al. (2023). "IconShop: Text-Guided Vector Icon Synthesis with Autoregressive Transformers". In: *ACM SIGGRAPH Asia*. URL: https://arxiv.org/abs/2304.14400.

Xing, Ximing et al. (2024). "SVGDreamer: Text Guided SVG Generation with Diffusion Model". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2312.16476.

— (2025). "Empowering LLMs to Understand and Generate Complex Vector Graphics". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2412.11102.

Xing, Ximing, Ziteng Xue, Yandong Guan, Jing Zhang, Dong Xu, and Qian Yu (2026). *Reason-SVG: Enhancing Structured Reasoning for Vector Graphics Generation with Reinforcement Learning*. arXiv: 2505.24499 [cs.CV]. URL: https://arxiv.org/abs/2505.24499.

Yang, An, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang,

Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu (2025). *Qwen3 Technical Report*. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.

Yang, An, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu (2025). *Qwen2.5 Technical Report*. arXiv: 2412.15115 [cs.CL]. URL: https://arxiv.org/abs/2412.15115.

Yang, Yiying et al. (2025). "OmniSVG: A Unified Scalable Vector Graphics Generation Model". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2504.06263.